Haley Candia Perez

10/15/2026

CS-340

Project Two

About the Project/Project Title

Grazioso Salvare Rescue Dog Dashboard

Client: Grazioso Salvare, an international company that trains rescue animals

Developer: Global Rain, a custom software engineering company

Grazioso Salvare identifies dogs that are good candidates for search-and-rescue training. Once trained, these dogs find and help rescue humans or other animals, often in life-threatening conditions. To find candidates, Grazioso Salvare has partnered with a nonprofit agency that operates five animal shelters in the region around Austin, Texas. The agency shares outcome data from those shelters.

Global Rain was contracted to build a full stack application that lets Grazioso Salvare work with this data. In Project One, Global Rain delivered a secured MongoDB database and a reusable CRUD Python module. Project Two completes the application with a client-facing web dashboard. Grazioso Salvare staff will use it to find and categorize dogs for training. Project Two delivers:

- **An interactive dashboard:** ProjectTwoDashboard.ipynb builds a web application with Dash. Staff can filter the shelter data by rescue type and view the results in a data table, a pie chart, and a geolocation map.
- **Rescue type queries:** Each filter option runs a MongoDB query that matches the breed, sex, and age range Grazioso Salvare prefers for that type of rescue: water rescue, mountain or wilderness rescue, and disaster or individual tracking.
- **The CRUD Python module:** CRUD_Python_Module.py, developed in Project One, serves as the data layer for the dashboard. The dashboard runs every query through its read method.

At Grazioso Salvare's request, the code is open source so that similar organizations can use and adapt it.

Motivation

The shelter data set contains 10,000 animal outcome records, covering dogs, cats, and other animals of every age and breed. Grazioso Salvare only needs a small, specific slice of it, and that slice is different for each type of rescue:

- **Age:** Search-and-rescue training is most effective for young dogs, so each rescue type has a preferred age range.

- **Breed:** Certain breeds are better suited to particular kinds of rescue. For example, Newfoundlands and Chesapeake Bay Retrievers are preferred for water rescue, and Bloodhounds are preferred for tracking a specific person by scent.
- **Sex:** Each rescue type also has a preferred sex, such as intact female for water rescue.

After Project One, the data was stored securely and could be searched, but only by writing Python queries by hand. Grazioso Salvare's trainers are not database developers, so that approach was slow and error-prone for them. This project solves the problem with a dashboard. Staff choose a rescue type with a single click, and the dashboard runs the correct query automatically. It shows the matching dogs in a table, a breakdown of their breeds, and the location of a selected dog on a map. The interface is designed to be user-friendly and intuitive, which reduces user errors and training time. Staff can find the best training candidates without knowing MongoDB or Python.

Getting Started

Follow these steps in order to import the data set, create the aacuser account, and run the dashboard. Screenshots of the working dashboard are in the Tests section.

Part 1: Import the data set

1. Open a terminal and move into the directory that contains the data set:
    a. cd~/workspace/datasets
2. Confirm that aac_shelter_outcomes.csv is there:
    a. ls
3. Import the CSV into the aac database and animals collection with mongoimport:
    a. mongoimport –db aac –collection animals –type csv –headerline –drop./aac_shelter_outcomes.csv
    b. The –type csv and –headerline options use the first row of the CSV as field names, and –drop removes any existing animals collection first so the import can be rerun safely.
4. Check the output for the success message. All 10,000 documents should import with 0 failures.

Part 2: Create the aacuser account

5. Open the mongo shell:
    a. mongosh
6. Switch to the admin authentication database. The user is created here even though its only role is on aac:
    a. use admin
7. Create aacuser with read and write access to the aac database only. Replace <your password> with a strong password and keep it somewhere safe, because the dashboard needs it:
    a. db.createUser({user: "aacuser", pwd: "<your password>", roles: [{role: "readWrite", db: "aac"}]})
8. Exit the shell:

a. exit

Part 3: Prepare the project files

9. Install the required libraries if they are not already installed (see Installation).
10. Place the following files in the same directory:
    a. ProjectTwoDashboard.ipynb
    b. CRUD_Python_Module.py
    c. Grazioso Salvare Logo.png
11. Open ProjectTwoDashboard.ipynb and make sure the username and password variables match your aacuser account:
    a. username = “aacuser”
    b. password = “<your password>”

Part 4: Run the dashboard

12. Open ProjectTwoDashboard.ipynb in Jupyter Lab and select Kernel -> Restart Kernel and Run All Cells.
13. When the cell finishes, select the link it prints to open the dashboard in a browser.
14. If port 8050 is already in use from an earlier run, restart the kernel, or change the last line of the notebook to app.run_server(port=8051).

Installation

The project was developed in the Codio Virtual Lab with the following tools:

Tools	Version	Purpose
MongoDB	7.0.21	Document database that stores the shelter data (the model)
Mongosh	2.5.3	MongoDB shell used for setup and user management
Python	3.11.2	Language for the CRUD module and dashboard
PyMongo	4.13.2	Official MongoDB driver for Python
Dash/Jupyter Dash	2.8.1	Web framework for the dashboard’s view and controller
Dash Leaflet	0.1.23	Interactive geolocation map
Plotly Express	5.14.1	Pie chart
Pandas	2.0.1	Converts query results into tables for the dashboard
Jupyter Lab	-	Environment for running the dashboard notebook

MongoDB (the model) stores the shelter data as documents in a collection. It is a good fit for this project because:

- Each animal record is a JSON-like document that maps directly to a Python dictionary, so query results pass straight into pandas and the dashboard with no translation layer.
- Queries are written as Python dictionaries using operators such as $in, $gte, and $lte. That made it simple to turn each row of Grazioso Salvare's rescue type table into a query.
- Its flexible schema handles shelter records whose fields vary from animal to animal.
- It supports user authentication with role-based access, so the dashboard connects as aacuser with only the permissions it needs.
- Through PyMongo, the official driver, MongoDB works directly in Jupyter and in Python web frameworks such as Dash, so the same CRUD module from Project One powers the dashboard.

Dash (the view and controller) is an open-source Python framework from Plotly for building interactive web applications without writing HTML or JavaScript. It was chosen because:

- It provides both the view and the controller in the Model-View-Controller design pattern. The layout, built from html, dcc (Dash Core Components), and dash_table components, defines what the user sees. Callbacks define how the dashboard responds when the user interacts with it. A callback is a Python function linked to specific inputs and outputs.
- It is written entirely in Python, so it works directly with the CRUD module, PyMongo, and pandas.
- It includes ready-made interactive components, such as radio buttons and a data table with built-in sorting, filtering, and pagination.
- It integrates with Plotly for charts and Dash leaflet for maps.
- JupyterDash allows the dashboard to run inside Jupyter Lab in Codio.

Codio provided MongoDB, Python, and the AAC data set preinstalled, so no local setup was required. To run the dashboard in another environment, you need:

- A running MongoDB instance with the aac database and animals collection
- An aacuser account with read and write access to aac
- The libraries above, which can be installed with:
    - pip install pymongo pandas dash jupyter-dash dash-leaflet plotly

Usage

The dashboard follows the Model-View-Controller (MVC) design pattern :

- **Model :** The aac database and animals collection in MongoDB, accessed through the AnimalShelter class in CRUD_Python_Module.py.
- **View :** The dashboard layout, made up of the logo, the unique identifier, the filter options, the data table, the pie chart, and the geolocation map.

- **Controller:** The Dash callbacks, which run MongoDB queries through the CRUD module when the user selects a filter, then update the table, chart, and map.

Dashboard components :

Components	Description
Logo	The Grazioso Salvare logo, which links to the client's home page (www.snhu.edu)
Unique Identifier	"Dashboard created by Haley Candia Perez," shown below the title
Filter options	Radio buttons for Water Rescue, Mountain or Wilderness Rescue, Disaster or Individual Tracking, and Reset
Data table	Shows the animals that match the selected filter. It supports sorting by column, filtering by typing under a column header, pagination (10 rows per page), and single-row selection
Pie chart	Shows the breeds of the animals currently in the data table. The 10 most common breeds are shown, and the rest are grouped into "Other"
Geolocation map	Places a marker at the location of the selected animal. Hovering over the marker shows its breed, and selecting it shows the animal's name

Filter options : Each filter runs a query based on the Rescue Type and Preferred Dog Breeds Table in the Dashboard Specifications Document :

Filter	Preferred Breeds	Preferred Sex	Training Age
Water Rescue	Labrador Retriever Mix, Chesapeake Bay Retriever, Newfoundland	Intact Female	26 to 156 weeks
Mountain or Wilderness Rescue	German Shepherd, Alaskan Malamute, Old English Sheepdog, Siberian Husky, Rottweiler	Intact Male	26 to 156 weeks
Disaster or Individual Tracking	Doberman Pinscher, German Shepherd, Golden Retriever, Bloodhound, Rottweiler	Intact Male	20 to 300 weeks
Reset	All records	-	-

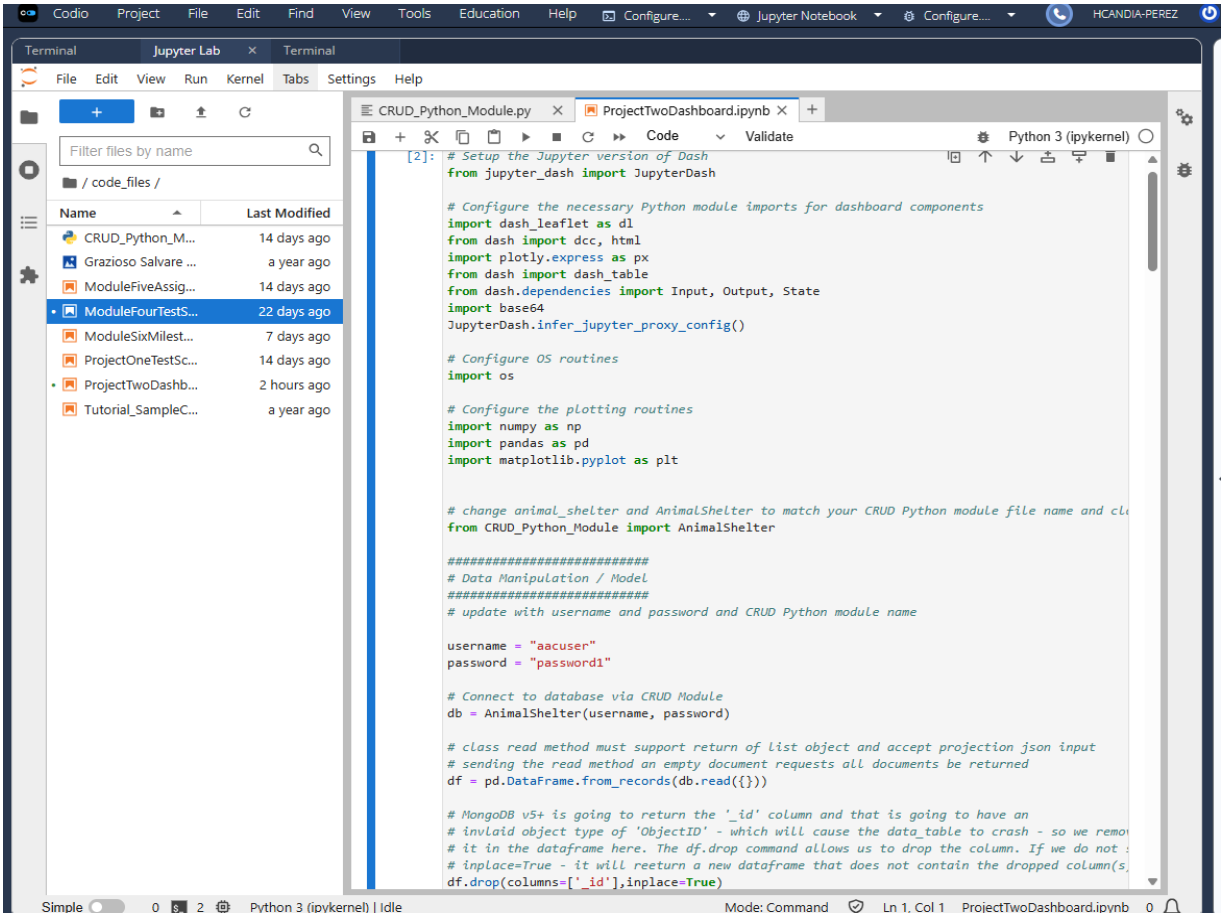
Callbacks :

- Update_dashboard : runs when a filter is selected. It builds the matching MongoDB query, sends it to the CRUD module's read method, removes the _id column, and sends the results to the data table. Reset sends an empty query, which returns every record.
- Update_graphs : runs whenever the data table changes. It counts the breeds in the table's current data and builds the pie chart.
- Update_map : runs whenever the data table or the selected row changes. It places a map marker at the selected animal's latitude and longitude.
- Update_styles : highlights a column in the data table when the user selects it.

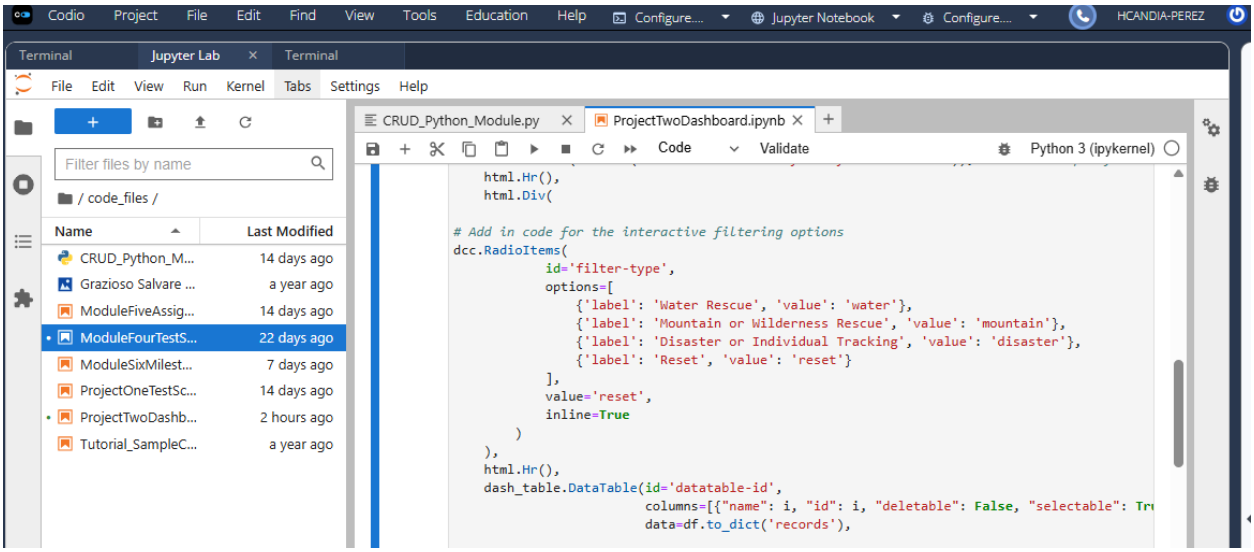
Code Example

The following examples from ProjectTwoDashboard.ipynb show how the dashboard uses the CRUD module.

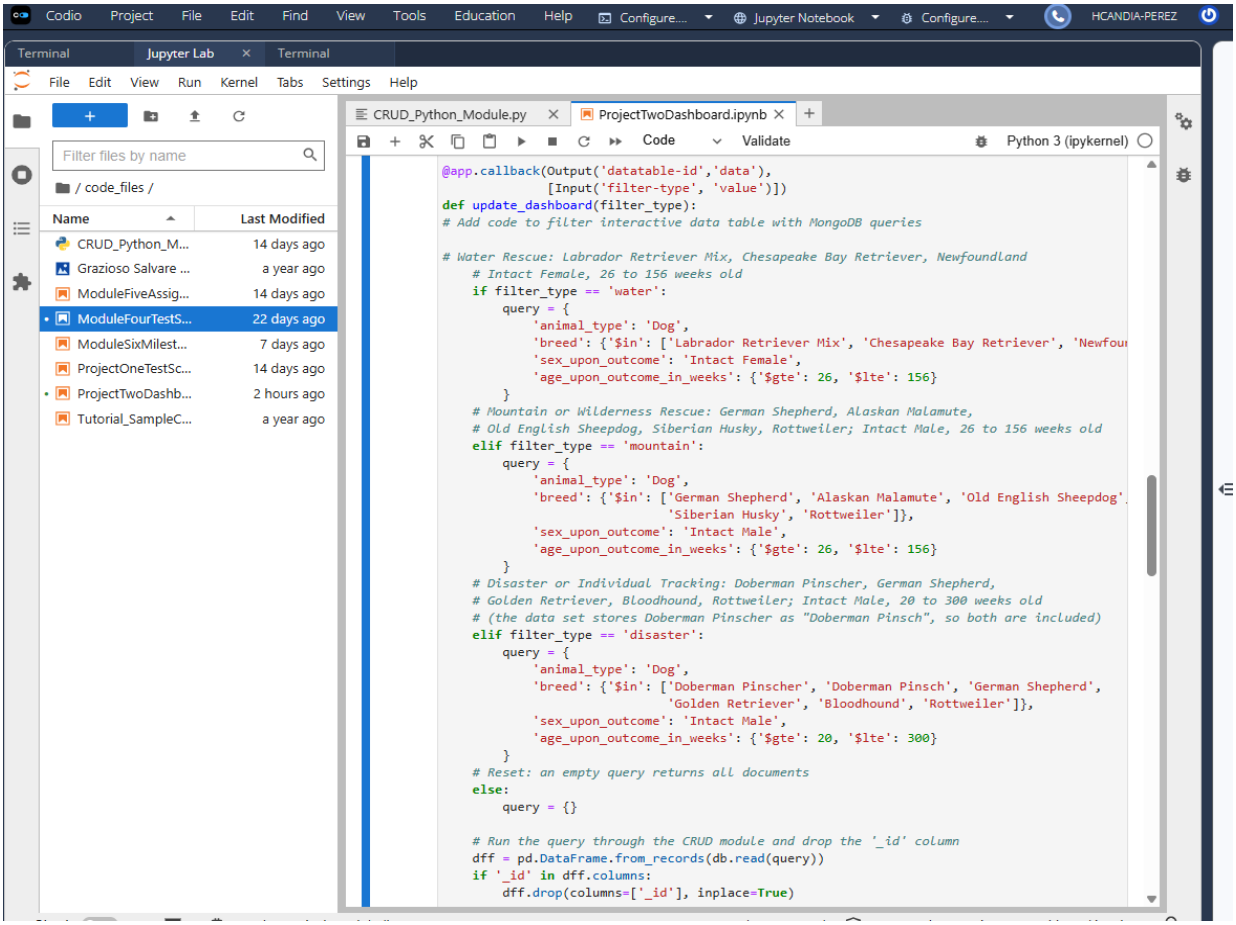
First, import the AnimalShelter class, connect to MongoDB as aacuser, and load every record for the unfiltered view :

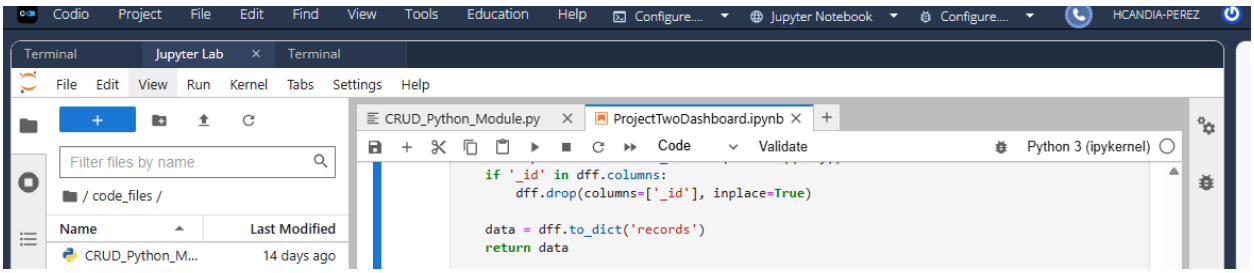


Define the filter options as radio buttons :

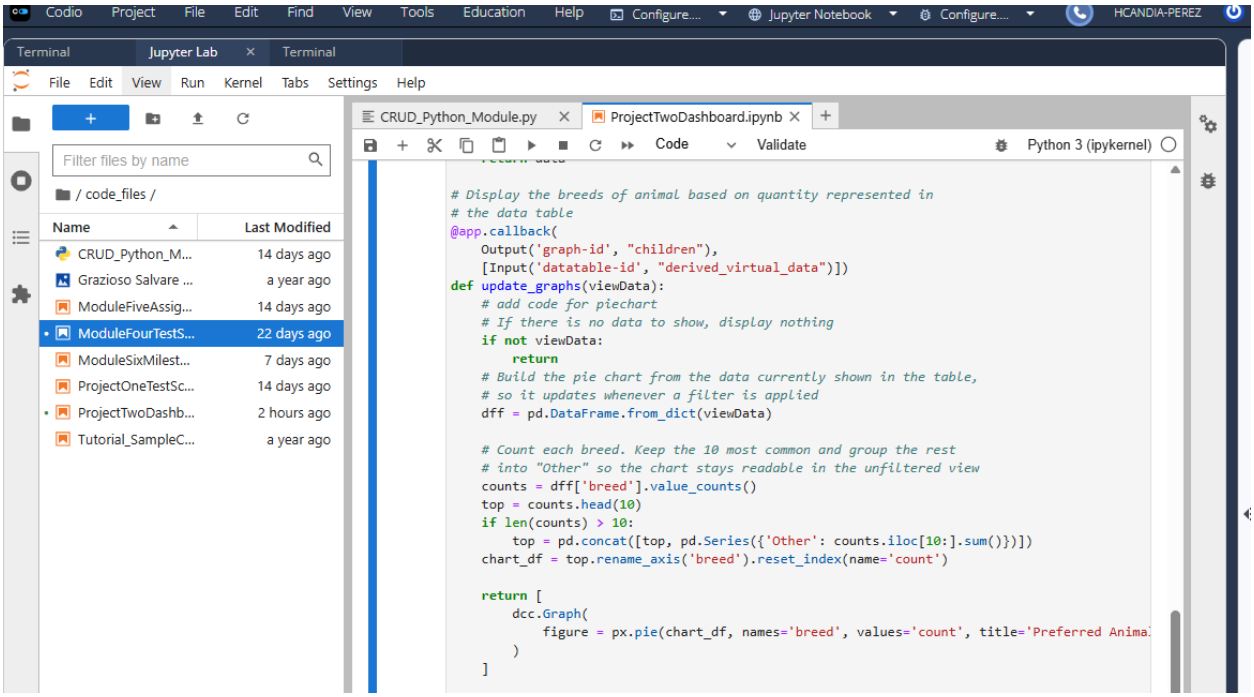


Build the query for the selected rescue type and run it through the CRUD module :





Build the pie chart from the data currently shown in the table :



Tests

To test the dashboard, open ProjectTwoDashboard.ipynb in Jupyter Lab, select Kernel -> Restart Kernel and Run All Cells, and open the dashboard link. Each filter was then selected in turn to confirm that the data table, pie chart, and map all update together. The tests verified the following :

- **Starting state :** The dashboard loaded with the logo, unique identifier, filter options, the unfiltered data table, the pie chart, and the map.

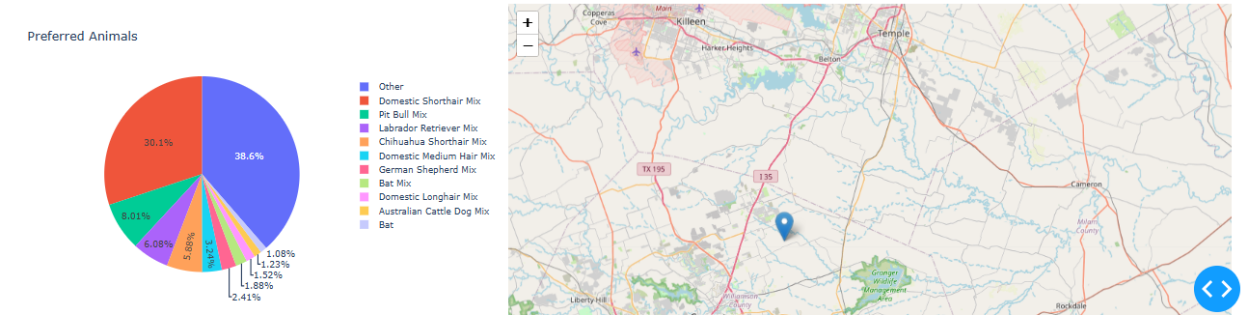
CS-340 Dashboard

Dashboard created by Haley Candia Perez

Water Rescue | Mountain or Wilderness Rescue | Disaster or Individual Tracking | Reset

rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name	outcome_subtype	outcome_type	sex_upon_outcome	location_lat
filter da													
3	2 years	A716330	Dog	Chihuahua Shorthair Mix	Brown/White	2013-11-18	2015-12-28 18:43:00	2015-12-28T18:43:00	Frank		Adoption	Neutered Male	30.7595748121648
9	3 years	A720214	Dog	Labrador Retriever Mix	Red/White	2013-02-04	2016-02-11 12:41:00	2016-02-11T12:41:00	Blessing		Adoption	Spayed Female	30.3870648199411
10	3 months	A664290	Cat	Domestic Shorthair Mix	Tortie	2013-09-01	2013-12-08 14:58:00	2013-12-08T14:58:00	*Taylor		Adoption	Spayed Female	30.7583105481048
11	1 year	A721199	Dog	Dachshund Wirehair Mix	Tan/White	2015-02-23	2016-02-27 17:49:00	2016-02-27T17:49:00	Belle		Adoption	Spayed Female	30.7290272761146
2	1 year	A725717	Cat	Domestic Shorthair Mix	Silver Tabby	2015-05-02	2016-05-06 10:49:00	2016-05-06T10:49:00		SCRP	Transfer	Spayed Female	30.6525984560228
1	3 years	A746874	Cat	Domestic Shorthair Mix	Black/White	2014-04-10	2017-04-11 09:00:00	2017-04-11T09:00:00		SCRP	Transfer	Neutered Male	30.5066578739455
12	1 year	A664843	Dog	Pit Bull Mix	Brown/White	2013-06-09	2014-08-18 17:24:00	2014-08-18T17:24:00	Sherlock	Partner	Transfer	Neutered Male	30.4515549397366
5	2 years	A691584	Dog	Labrador Retriever Mix	Tan/White	2012-11-06	2015-05-30 13:48:00	2015-05-30T13:48:00	Luke		Return to Owner	Neutered Male	30.7104815618433
13	1 year	A700408	Cat	Domestic Shorthair Mix	Brown Tabby/White	2014-04-13	2015-04-15 13:34:00	2015-04-15T13:34:00	Nyla		Return to Owner	Spayed Female	30.4101154527976
14	2 years	A742287	Dog	Boxer/Bullmastiff	Brown Brindle/White	2015-01-18	2017-02-11 12:30:00	2017-02-11T12:30:00	*Kawhi		Adoption	Neutered Male	30.4551148649096

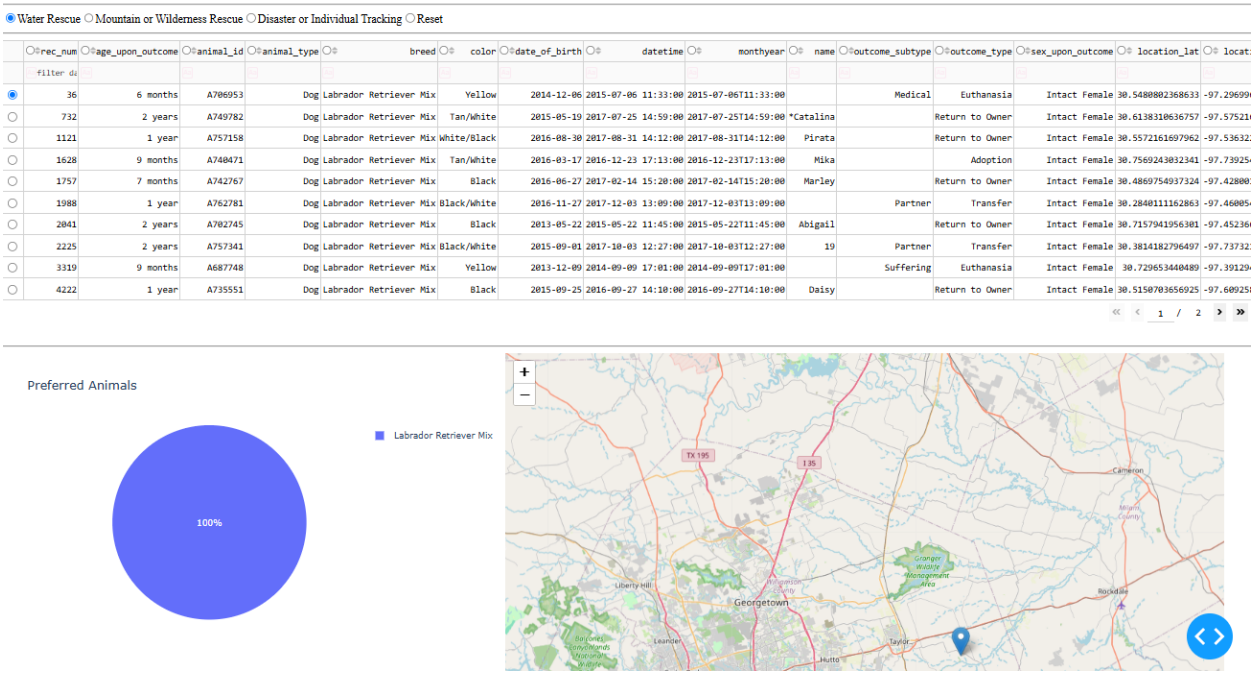
1 / 1001



- **Water Rescue :** The table showed only intact female Labrador Retriever Mixes, Chesapeake Bay Retrievers, and Newfoundlands between 26 and 156 weeks old. The pie chart and map updated to match.

CS-340 Dashboard

Dashboard created by Haley Candia Perez



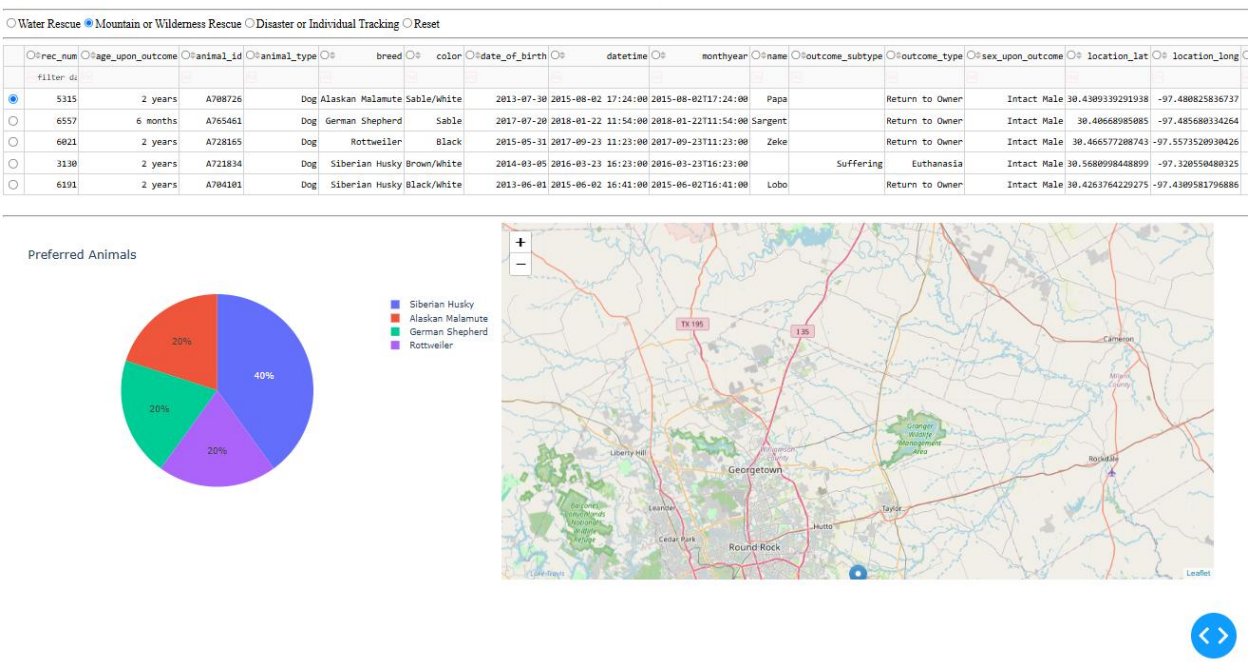
	rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name	outcome_subtype	outcome_type	sex_upon_outcome	location_lat	locat
	filter da														
◉	36	6 months	A706953	Dog	Labrador Retriever Mix	Yellow	2014-12-06	2015-07-06 11:33:00	2015-07-06T11:33:00		Medical	Euthanasia	Intact Female	30.5480802368633	-97.29699
○	732	2 years	A749782	Dog	Labrador Retriever Mix	Tan/White	2015-05-19	2017-07-25 14:59:00	2017-07-25T14:59:00	*Catalina		Return to Owner	Intact Female	30.6138310636757	-97.57521
○	1121	1 year	A757158	Dog	Labrador Retriever Mix	White/Black	2016-08-30	2017-08-31 14:12:00	2017-08-31T14:12:00	Pirata		Return to Owner	Intact Female	30.5572161697962	-97.53632
○	1628	9 months	A740471	Dog	Labrador Retriever Mix	Tan/White	2016-03-17	2016-12-23 17:13:00	2016-12-23T17:13:00	Mika		Adoption	Intact Female	30.7569243032341	-97.73925
○	1757	7 months	A742767	Dog	Labrador Retriever Mix	Black	2016-06-27	2017-02-14 15:20:00	2017-02-14T15:20:00	Marley		Return to Owner	Intact Female	30.486975493732 4	-97.42800
○	1988	1 year	A762781	Dog	Labrador Retriever Mix	Black/White	2016-11-27	2017-12-03 13:09:00	2017-12-03T13:09:00		Partner	Transfer	Intact Female	30.2840111162863	-97.46005
○	2041	2 years	A702745	Dog	Labrador Retriever Mix	Black	2013-05-22	2015-05-22 11:45:00	2015-05-22T11:45:00	Abigail		Return to Owner	Intact Female	30.7157941956301	-97.45236
○	2225	2 years	A757341	Dog	Labrador Retriever Mix	Black/White	2015-09-01	2017-10-03 12:27:00	2017-10-03T12:27:00	19	Partner	Transfer	Intact Female	30.3814182796497	-97.73732
○	3319	9 months	A687748	Dog	Labrador Retriever Mix	Yellow	2013-12-09	2014-09-09 17:01:00	2014-09-09T17:01:00		Suffering	Euthanasia	Intact Female	30.729653440489	-97.39129
○	4222	1 year	A735551	Dog	Labrador Retriever Mix	Black	2015-09-25	2016-09-27 14:10:00	2016-09-27T14:10:00	Daisy		Return to Owner	Intact Female	30.5150703656925	-97.60925

« ‹ 1 / 2 › »

- **Mountain or Wilderness Rescue :** The table showed only intact male German Shepherds, Alaskan Malamutes, Old English Sheepdogs, Siberian Huskies, and Rottweilers between 26 and 156 weeks old. The pie chart and map updated to match.

CS-340 Dashboard

Dashboard created by Haley Candia Perez



	rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name	outcome_subtype	outcome_type	sex_upon_outcome	location_lat	location_long
◉	5315	2 years	A708726	Dog	Alaskan Malamute	Sable/White	2013-07-30	2015-08-02 17:24:00	2015-08-02T17:24:00	Papa		Return to Owner	Intact Male	30.4309339291938	-97.480825836737
○	6557	6 months	A765461	Dog	German Shepherd	Sable	2017-07-20	2018-01-22 11:54:00	2018-01-22T11:54:00	Sargent		Return to Owner	Intact Male	30.40668985085	-97.485680334264
○	6021	2 years	A728165	Dog	Rottweiler	Black	2015-05-31	2017-09-23 11:23:00	2017-09-23T11:23:00	Zeke		Return to Owner	Intact Male	30.466577208743	-97.5573520930426
○	3130	2 years	A721834	Dog	Siberian Husky	Brown/White	2014-03-05	2016-03-23 16:23:00	2016-03-23T16:23:00		Suffering	Euthanasia	Intact Male	30.5680998448899	-97.320550480325
○	6191	2 years	A704101	Dog	Siberian Husky	Black/White	2013-06-01	2015-06-02 16:41:00	2015-06-02T16:41:00	Lobo		Return to Owner	Intact Male	30.4263764229275	-97.4309581796886

- Disaster or Individual Tracking: The table showed only intact male Doberman Pinschers, German Shepherds, Golden Retrievers, Bloodhounds, and Rottweilers between 20 and 300 weeks old. The pie chart and map updated to match.



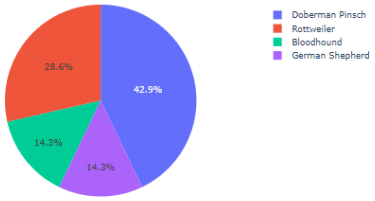
CS-340 Dashboard

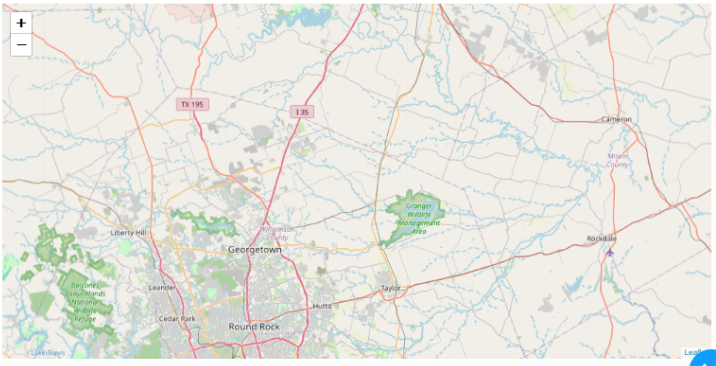
Dashboard created by Haley Candia Perez

○ Water Rescue ○ Mountain or Wilderness Rescue ◉ Disaster or Individual Tracking ○ Reset

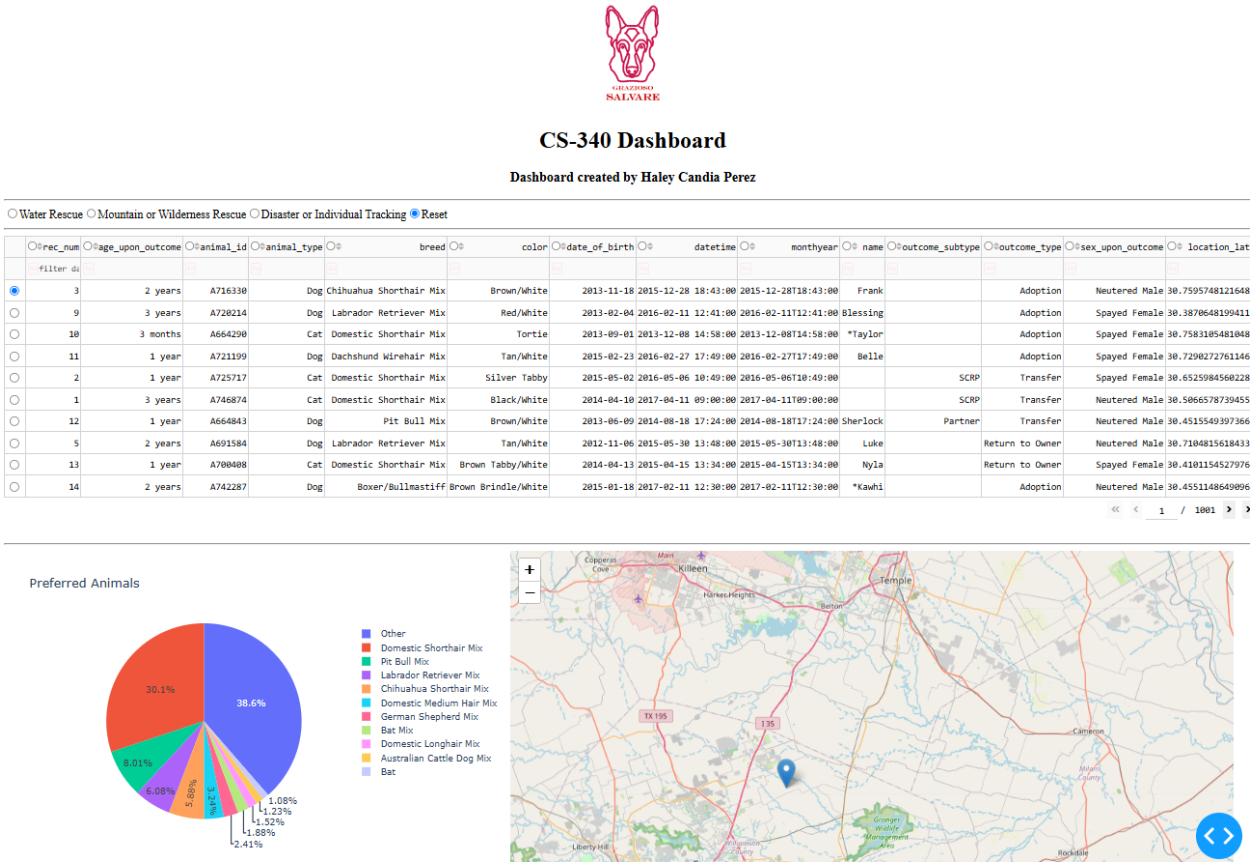
	rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name	outcome_subtype	outcome_type	sex_upon_outcome	location_lat	location_long
	filter d														
◉	3767	4 years	A712291	Dog	Bloodhound	Red	2011-09-20	2015-09-22 15:43:00	2015-09-22T15:43:00	Boomer		Return to Owner	Intact Male	30.2709983761287	-97.5923916912722
○	622	2 years	A741441	Dog	Doberman Pinsch	Blue/Tan	2015-01-04	2017-01-04 14:02:00	2017-01-04T14:02:00	Deeohgee		Return to Owner	Intact Male	30.6176290949888	-97.4249284729106
○	4723	8 months	A731795	Dog	Doberman Pinsch	Black/Tan	2015-11-15	2016-08-06 21:42:00	2016-08-06T21:42:00	Negro		Return to Owner	Intact Male	30.5663297388775	-97.6449426083984
○	7840	1 year	A654132	Dog	Doberman Pinsch	Black/Red	2012-07-16	2014-04-18 11:11:00	2014-04-18T11:11:00	Zorro	Partner	Transfer	Intact Male	30.760756173511	-97.3695237014692
○	6557	6 months	A765461	Dog	German Shepherd	Sable	2017-07-20	2018-01-22 11:54:00	2018-01-22T11:54:00	Sargent		Return to Owner	Intact Male	30.40668985085	-97.485680334264
○	2987	4 years	A694614	Dog	Rottweiler	Black/Brown	2011-01-01	2015-01-01 14:25:00	2015-01-01T14:25:00	Striker		Return to Owner	Intact Male	30.329873203611	-97.5492968638502
○	6021	2 years	A728165	Dog	Rottweiler	Black	2015-05-31	2017-09-23 11:23:00	2017-09-23T11:23:00	Zeke		Return to Owner	Intact Male	30.466577208743	-97.5573520930426

Preferred Animals





- **Reset :** All widgets returned to their original, unfiltered state.



rec_num	age_upon_outcome	animal_id	animal_type	breed	color	date_of_birth	datetime	monthyear	name	outcome_subtype	outcome_type	sex_upon_outcome	location_lat
3	2 years	A716330	Dog	Chihuahua Shorthair Mix	Brown/White	2013-11-18	2015-12-28 18:43:00	2015-12-28T18:43:00	Frank		Adoption	Neutered Male	30.7595748121648
9	3 years	A720214	Dog	Labrador Retriever Mix	Red/White	2013-02-04	2016-02-11 12:41:00	2016-02-11T12:41:00	Blessing		Adoption	Spayed Female	30.3870648199411
10	3 months	A664290	Cat	Domestic Shorthair Mix	Tortie	2013-09-01	2013-12-08 14:58:00	2013-12-08T14:58:00	*Taylor		Adoption	Spayed Female	30.7583105481048
11	1 year	A721199	Dog	Dachshund Wirehair Mix	Tan/White	2015-02-23	2016-02-27 17:49:00	2016-02-27T17:49:00	Belle		Adoption	Spayed Female	30.7290272761146
2	1 year	A725717	Cat	Domestic Shorthair Mix	Silver Tabby	2015-05-02	2016-05-06 10:49:00	2016-05-06T10:49:00		SCRP	Transfer	Spayed Female	30.6525984560228
1	3 years	A746874	Cat	Domestic Shorthair Mix	Black/White	2014-04-10	2017-04-11 09:00:00	2017-04-11T09:00:00		SCRP	Transfer	Neutered Male	30.5066578739455
12	1 year	A664843	Dog	Pit Bull Mix	Brown/White	2013-06-09	2014-08-18 17:24:00	2014-08-18T17:24:00	Sherlock	Partner	Transfer	Neutered Male	30.4515549397366
5	2 years	A691584	Dog	Labrador Retriever Mix	Tan/White	2012-11-06	2015-05-30 13:48:00	2015-05-30T13:48:00	Luke		Return to Owner	Neutered Male	30.7104815618433
13	1 year	A700408	Cat	Domestic Shorthair Mix	Brown Tabby/White	2014-04-13	2015-04-15 13:34:00	2015-04-15T13:34:00	Nyla		Return to Owner	Spayed Female	30.4101154527976
14	2 years	A742287	Dog	Boxer/Bullmastiff	Brown Brindle/White	2015-01-18	2017-02-11 12:30:00	2017-02-11T12:30:00	*Kawhi		Adoption	Neutered Male	30.4551148649096

Screenshots

Screenshots appear throughout this document next to the steps they demonstrate:

- Dashboard code : Code Example
- Starting state and all four filters : Tests

Steps taken to Complete the Project

1. Reviewed the Dashboard Specifications Document, including the required widgets, the branding requirements, and the Rescue Type and Preferred Dog Breeds Table.
2. Imported the CRUD Python module from Project One into the dashboard notebook and connected to MongoDB as aacuser.
3. Built the unfiltered data table with a “retrieve all” query, db.read({}), and added sorting, filtering, pagination, and row selection to make it easier for the client to use.
4. Wrote a MongoDB query for each rescue type, matching the preferred breeds, sex, and age range.
5. Added radio buttons as the filter options and wrote a callback that runs the matching query and updates the data table.
6. Added a pie chart and a geolocation map that update from the data table’s current data.

7. Added the Grazioso Salvare logo, with a link to the client's home page, and a unique identifier with the developer's name.
8. Tested each filter and captured screenshots of each state.
9. Wrote this README to document the project and provide instructions for reproducing it.

Challenges

- **ObjectID serialization :** MongoDB returns an _id field of type ObjectID, which the Dash data table cannot display. The _id column is dropped from the data before it reaches the table, both when the dashboard loads and after every filter query.
- **Inconsistent breed names :** The data set stores "Doberman Pinscher" as "Doberman Pinsch." Both spellings are included in the Disaster or Individual Tracking query so those dogs are not missed.
- **Startup error:** When the dashboard first loaded, it displayed the error "'NoneType' object is not iterable." No column was selected yet, so the column-highlight callback tried to loop over an empty value. Setting selected_columns=[] on the data table resolved it, because the table now starts with a valid empty list.
- **Unreadable pie chart:** The unfiltered data contains hundreds of breeds, which produced hundreds of tiny slices with overlapping labels. Showing the 10 most common breeds and grouping the rest into "Other" resolved it.
- **Pie chart not updating:** The starter code's sample pie chart used the original, unfiltered DataFrame, so it would not change when a filter was selected. It was changed to use the data table's current data (derived_virtual_data), so it updates with every filter.

Roadmap/Features

The dashboard now provides complete filtering by rescue type, with a data table, pie chart, and geolocation map that update together. Possible future features include :

- Showing all filtered dogs on the map at once, rather than only the selected dog
- Adding filters for shelter location or outcome date
- Letting staff adjust the age range or add breeds without changing the code

Resources

- MongoDB documentation: https://www.mongodb.com/docs/
- PyMongo documentation: https://pymongo.readthedocs.io/
- Dash documentation: https://dash.plotly.com/
- Dash Core Components: https://dash.plotly.com/dash-core-components
- Dash DataTable: https://dash.plotly.com/datatable
- Dash Leaflet: https://www.dash-leaflet.com/
- Plotly Express: https://plotly.com/python/plotly-express/
- pandas: https://pandas.pydata.org/docs/

- Jupyter: https://jupyter.org/
- Austin Animal Center Outcomes data: https://data.austintexas.gov/

Contact

Haley Candia Perez